1. What in nopCommerce's design helped or hindered your instrumentation work?

What helped?

- **Dependency Injection (DI)** - nopCommerce heavily relies on DI to resolve components at runtime. This loosely coupled architectural style was immensely beneficial for observability, as it allowed us to inject OpenTelemetry libraries, `Meter` singletons, and `Tracer` instances across different services without violating the Single Responsibility Principle or heavily modifying existing constructor signatures.
- **Clear Separation of Concerns** - The codebase is well-structured into distinct service layers (e.g., `OrderProcessingService`, `PaymentService`, `ProductService`). This strict boundary enforcement provided natural, logical start and end points for our trace spans, ensuring our telemetry closely mirrored the actual domain boundaries.
- **Centralized Event Publisher** - Internally, nopCommerce utilizes an `IEventPublisher` pattern (acting as an in-memory event bus). Rather than scattering instrumentation code across dozens of entity services, this design provided a highly centralized interception point. We were able to capture and observe domain events globally without polluting the core business logic.

What hindered?

- **Monolithic Database Locking** - The application relies on a single relational database coupled with aggressive application-level locking (Mutex locking during checkout) to prevent inventory overselling. While necessary for consistency in this architecture, this tightly coupled paradigm severely hindered our observability efforts. It artificially obscured the true source of latency: requests from concurrent users were not slow execution-wise, but were instead blocked in memory waiting for thread locks. This forced us to rethink metric placement, as we had to instrument the outermost boundary of the operation to capture the true user-facing latency, rather than the execution time of the method itself.

2. If you were making architectural decisions on this project going forward, what would you change to make it more observable — and at what cost?

- **Extracting Inventory Management to an Asynchronous Service**: Moving the inventory management logic to an observable message queue, rather than relying on the current inline SQL row-locking mechanism, would flatten latency spikes during moments of high concurrency. It would also allow us to monitor "queue length" and "message processing time" as precise metrics, rather than simply watching HTTP requests time out. Despite these advantages, this requires a massive architectural rewrite toward Eventual Consistency, significantly increasing system complexity and introducing business risks such as product overselling.

- **Correlating Application Logs with OpenTelemetry Traces**: As it stands, nopCommerce writes application errors and warnings directly to a SQL relational `Log` table. While distributed tracing was implemented, these two pillars of observability are completely divorced. If a user checkout fails and creates a slow trace in Jaeger, there's a need to manually search the database `Log` table by timestamp to figure out why it failed. Replacing the custom database logging with a structured logging framework integrated directly with OpenTelemetry, would automatically inject the `TraceId` and `SpanId` into every log entry. When an order fails, the exact error message would be permanently attached to the specific span in Jaeger. The cost here involves a significant refactoring of the `ILogger` implementation and migrating away from the legacy SQL log storage.

3. Where did you have to make a surgical change to the existing code? Why was it necessary, and how did you minimise the impact?

- **Centralized Telemetry (`NopMetrics` & `NopTracing`)**: These two static classes were explicitly created to bridge the nopCommerce core and the OpenTelemetry API. Rather than handling exports, they act as global repositories defining the central `Meter` and `Tracer` singletons. This architectural choice minimized impact across the application, as individual service classes only needed to interact with these global definitions rather than instantiating and managing their own telemetry logic.

- **Global Event Interception (`EventPublisher`)**: The core event publishing mechanism was wrapped with an active OpenTelemetry Trace Span. By intercepting events here, every single domain event system-wide automatically generated trace spans. This eliminated the need to manually inject tracing logic into dozens of individual event consumers, keeping the core domain logic pristine.

- **Startup Composition (`Program.cs`)**: The application's entry point was modified to inject the OpenTelemetry SDK, configure the OTLP Exporters to point to our Docker Collector, and enable automatic SQL Client instrumentation. By utilizing the native `IServiceCollection` extension methods during startup, we successfully wired up the entire observability pipeline without needing to alter a single Web Controller or API endpoint.

- **Service Layer Instrumentation (`OrderProcessingService` & `ProductService`)**: Surgical modifications were required in the service layer to capture accurate business metrics. For example, in `OrderProcessingService`, we deliberately wrapped the outermost boundary of the `PlaceOrderAsync` method to capture the true user-facing latency, including the time spent waiting for Database Mutex locks. By utilizing our centralized `NopMetrics`, these changes were restricted to 1-2 lines of code, minimizing the risk of introducing business logic errors.